CSS326 DATABASE PROGRAMMING LABORATORY · GROUP 10

All About Food!

ระบบจัดการออเดอร์ร้านอาหาร

เอกสารสรุประบบสำหรับทีม — ทำอะไรได้บ้าง มีระบบอะไร ข้อมูลไหลยังไง และโค้ดอยู่ตรงไหน

11
ตารางในฐานข้อมูล

5
ประเภทผู้ใช้ (ลูกค้า + 4 role)

5 + 1
trigger + stored procedure

60
test cases ผ่านทั้งหมด

สารบัญ

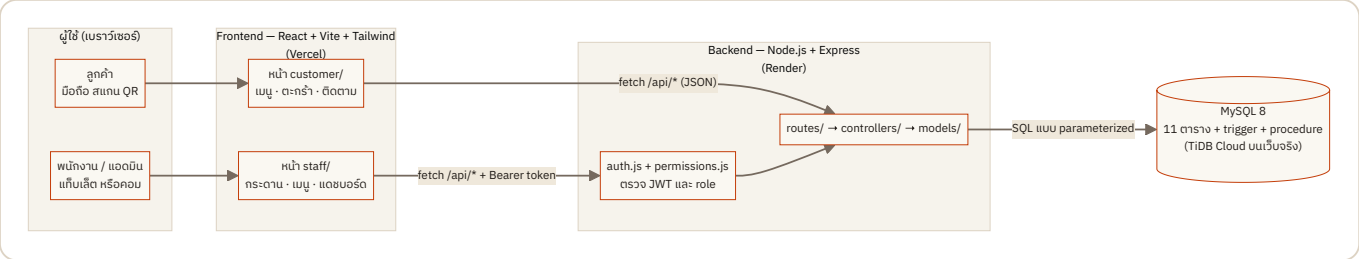
1 ภาพรวมและสถาปัตยกรรม
2 ผู้ใช้และสิทธิ์
3 ระบบย่อยทั้ง 10 ระบบ
4 Flow ฝั่งลูกค้า
5 วงจรชีวิตของออเดอร์
6 เบื้องหลัง 1 คำสั่ง (ข้อมูลไหลผ่านโค้ด)
7 Flow ชำระเงิน · login · ลบเมนู · จองโต๊ะ
8 หน้าจอจริงของระบบ
9 ฐานข้อมูล, trigger, procedure
10 โครงสร้างโค้ด: แก้อะไรที่ไฟล์ไหน
11 ความปลอดภัย
12 รัน, deploy, demo และสิ่งที่ยังไม่มี

เว็บ: restaurant-ahn-hclv.vercel.app · API: restaurant-ahn-1.onrender.com · โค้ด: github.com/White481/Restaurant_ahn · อัปเดต 31 ต.ค. 2569

01 ภาพรวมและสถาปัตยกรรม

ลูกค้าจองโต๊ะล่วงหน้าได้ และเมื่อมานั่งที่โต๊ะก็สแกน QR สั่งอาหารจากมือถือเองโดยไม่ต้อง login ออเดอร์เด้งขึ้นกระดานของพนักงานทันที ครัวทำ → พนักงานเสิร์ฟ → แคชเชียร์รับเงินและพิมพ์ใบเสร็จ ส่วนแอดมินจัดการเมนูและดูยอดขาย

ระบบแบ่งเป็น 3 ชั้น



ส่วน	เทคโนโลยี	อยู่ที่ไหน	หน้าที่
Frontend	React 18, Vite, Tailwind CSS v4, React Router	`frontend/` → Vercel	หน้าจอทั้งหมด ดึงข้อมูลผ่าน `services/api.js` ไฟล์เดียว
Backend	Node.js, Express, mysql2, jsonwebtoken, bcryptjs	`backend/` → Render	REST API, ตรวจสิทธิ์, คำนวณราคา, transaction
Database	MySQL 8 (ในเครื่อง) / TiDB Cloud (เว็บจริง)	`database/`	เก็บข้อมูล + กฎทางธุรกิจด้วย trigger และ procedure

> **ทำไมแยก 3 ชั้น:** หน้าเว็บไม่เคยคุยกับฐานข้อมูลตรงๆ ทุกอย่างต้องผ่าน API ซึ่งตรวจสิทธิ์และตรวจข้อมูลก่อนเสมอ ราคาอาหารก็ดึงจากฐานข้อมูลใหม่ทุกครั้ง ไม่เชื่อราคาที่หน้าเว็บส่งมา

ข้อมูลอัปเดตแบบ "เกือบ real-time" ยังไง

ใช้ **polling**: หน้าเว็บถาม API ซ้ำเป็นระยะ (ไฟล์ `hooks/usePolling.js`) และหยุดถามเมื่อแท็บถูกซ่อน

หน้า	ถามทุก	หยุดเมื่อ
กระดานออเดอร์ของพนักงาน	5 วินาที	แท็บถูกซ่อน · มีออเดอร์ใหม่จะมีเสียงเตือนและการ์ดกะพริบ
หน้าติดตามออเดอร์ของลูกค้า	4 วินาที	ออเดอร์เสร็จสิ้นหรือถูกยกเลิก
แดชบอร์ด / การจองของพนักงาน	30 วินาที	แท็บถูกซ่อน · เปลี่ยนวันที่แล้วโหลดใหม่ทันที

02 ผู้ใช้และสิทธิ์

ลูกค้าไม่ต้องมีบัญชี ส่วนพนักงานมี 4 role แต่ละ role เห็นเฉพาะปุ่มที่ตัวเองกดได้ และ backend ตรวจซ้ำทุกครั้ง ถึงจะยิง API ตรงโดยไม่ผ่านหน้าเว็บก็ทำไม่ได้

ทำอะไร	ลูกค้า	chef ครัว	waiter เสิร์ฟ	cashier แคชเชียร์	admin
ดูเมนู · สั่งอาหาร · ติดตามออเดอร์ตัวเอง · จองโต๊ะ	✓	✓	✓	✓	✓
ดูกระดานออเดอร์และประวัติ		✓	✓	✓	✓
ใหม่ → กำลังทำ → พร้อมเสิร์ฟ		✓			✓
พร้อมเสิร์ฟ → เสิร์ฟแล้ว			✓		✓
ยกเลิกออเดอร์ (ก่อนเสิร์ฟเท่านั้น)			✓		✓
รับชำระเงิน → เสร็จสิ้น + พิมพ์ใบเสร็จ			✓	✓	✓
จัดการการจอง (ยืนยัน · ลูกค้ามาถึง · ยกเลิก · ย้ายโต๊ะ)			✓		✓
ลบการจองถาวร					✓
จัดการเมนู (เพิ่ม/แก้/ซ่อน/ลบ)					✓
แดชบอร์ด ยอดขาย สต็อก					✓

กฎเพิ่มเติมของการเปลี่ยนสถานะ

- เดินหน้า**ทีละขั้น**เท่านั้น ข้ามขั้นหรือย้อนกลับไม่ได้ (API ตอบ `409`)
- role ที่ไม่มีสิทธิ์ได้ `403` · ไม่ได้ login ได้ `401`
- ออเดอร์ที่ **เสร็จสิ้น** หรือ **ยกเลิก** แล้ว แก้ไขไม่ได้อีก (ยอดขายจะได้ไม่เพี้ยน)
- "เสร็จสิ้น" ต้องผ่านการรับชำระเงินเสมอ ข้อมูลการจ่ายเงินจึงครบทุกออเดอร์

> **ตารางสิทธิ์อยู่ที่เดียว:** `backend/src/config/permissions.js` (ตัวจริงที่บังคับใช้) และสำเนาใน `frontend/src/components/status.js` (ใช้แค่ซ่อนปุ่ม) ถ้าจะเปลี่ยนสิทธิ์ต้องแก้ **ทั้งสองไฟล์**

บัญชีที่มีในระบบ

สร้างด้วย `npm run seed:users` ได้ 4 บัญชี: `admin`, `chef`, `waiter`, `cashier` รหัสผ่านตั้งใน `backend/.env` (ไม่ได้อยู่ในโค้ด) และเปลี่ยนได้ด้วย `npm run set:password -- <username>`

03 ระบบย่อยทั้ง 10 ระบบ

แต่ละการ์ดบอกว่าระบบนั้นทำอะไร ใครใช้ อยู่หน้าไหน เรียก API อะไร และแตะตารางไหนในฐานข้อมูล

1 สั่งอาหาร

- สแกน QR ของโต๊ะ (`/?table=3`) ระบบจำเลขโต๊ะไว้
- ค้นหาเมนู, กรองตามหมวด, ชื่อไทย/อังกฤษ, รูปเมนู
- ตะกร้า: เพิ่ม/ลดจำนวน (จำไว้แม้ refresh)
- ยืนยัน: ทานที่ร้าน/กลับบ้าน, วิธีจ่าย, โน้ตถึงครัว

ลูกค้า · `/` `/cart` · `GET /api/menus` `POST /api/orders` · orders, order_item

2 ติดตามออเดอร์

- แสดงขั้นตอน 5 ขั้น พร้อมสถานะปัจจุบัน
- อัปเดตเองทุก 4 วินาทีจนออเดอร์จบ
- เห็นแค่สถานะ ไม่เห็นข้อมูลอื่นของร้าน

ลูกค้า · `/order/:id` · `GET /api/orders/:id/status` · orders

10 จองโต๊ะล่วงหน้า

- ลูกค้ากรอกชื่อ เบอร์ วัน เวลา จำนวนคน ไม่ต้อง login
- ระบบเลือกโต๊ะเล็กสุดที่นั่งพอ กันจองซ้อน 2 ชม.
- พนักงาน: ยืนยัน · มาถึง · ยกเลิก · ย้ายโต๊ะ
- ยืนยันของวันนี้ → trigger โต๊ะเป็น "จองแล้ว"

ลูกค้า + admin/waiter · `/reserve` `/staff/reservations` · `/api/reservations` · reservation, customer

3 กระดานออเดอร์ (Kanban)

- 4 คอลัมน์: ใหม่ · กำลังทำ · พร้อมเสิร์ฟ · เสิร์ฟแล้ว
- ตัวจับเวลา: เหลือง > 10 นาที, แดง > 20 นาที
- ออเดอร์ใหม่มีเสียงเตือน + การ์ดกะพริบ
- ปุ่มบนการ์ดเปลี่ยนตาม role
- แท็บ "ประวัติ" ดูออเดอร์ที่จบแล้ว

พนักงานทุก role · `/staff` · `GET /api/orders` `PATCH /api/orders/:id`

4 รับชำระเงิน + ใบเสร็จ

- เงินสด (กรอกเงินที่รับ ระบบคิดเงินทอน) / บัตร / QR
- ใบเสร็จ 80 มม. แยก VAT 7% (ราคารวม VAT แล้ว)
- เลือกภาษาใบเสร็จ ไทย/English แยกจากภาษาหน้าเว็บ
- พิมพ์ซ้ำได้จากแท็บประวัติ

waiter, cashier, admin · `POST /api/orders/:id/pay` · orders (paid_by, paid_at, amount_received)

5 จัดการเมนู (CRUD)

- **C** เพิ่มเมนู + พรีวิวหน้าตาบนหน้าลูกค้า
- **R** ดูทั้งหมด รวมเมนูที่ซ่อน
- **U** แก้ไข, ซ่อน/แสดง
- **D** ลบ ถ้าเคยถูกสั่งแล้วจะซ่อนแทน

admin · `/staff/menu` · `GET /all` `POST` `PUT` `DELETE /api/menus` · menu_item

6 แดชบอร์ดยอดขาย

- รายได้ ออเดอร์ ยอดเฉลี่ย จำนวนจานของวันที่เลือก
- ตัวเลขยอดขายมาจาก stored procedure
- เมนูขายดี 5 อันดับ, ออเดอร์ตามสถานะ, ออเดอร์ล่าสุด

admin · `/staff/dashboard` · `GET /api/dashboard?date=` · `sp_sales_summary`

7 สต็อกวัตถุดิบ + สถานะโต๊ะ

- สั่งอาหาร → trigger ตัดวัตถุดิบตามสูตรอัตโนมัติ
- ของไม่พอ → ยกเลิกทั้งออเดอร์ แจ้งลูกค้า
- ยกเลิกออเดอร์ → คืนวัตถุดิบ
- โต๊ะเปลี่ยน ว่าง ↔ มีลูกค้า เองตามออเดอร์

ทำงานในฐานข้อมูล · ingredient, menu_item_ingredient, dining_table · แสดงบนแดชบอร์ด

8 Login + สิทธิ์

- login ได้ JWT อายุ 8 ชั่วโมง เก็บในเบราว์เซอร์
- รหัสผ่านเก็บเป็น bcrypt hash
- token หมดอายุ → เด้งกลับหน้า login เอง

พนักงาน · `/staff/login` · `POST /api/auth/login` · sys_user, employee

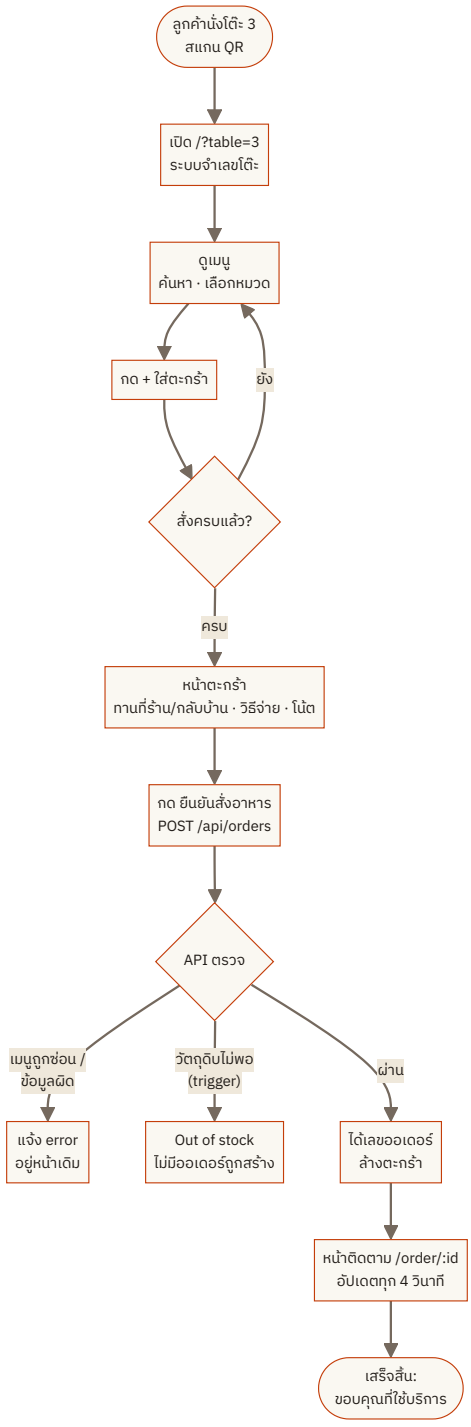
9 ภาษา · ธีม · มือถือ

- สลับ ไทย / EN ทุกหน้า (ข้อความทั้งหมดอยู่ใน `i18n/messages.js`)
- ฝั่งพนักงานมีโหมดมืด/สว่าง/ตามเครื่อง
- ใช้งานได้ทั้งมือถือ แท็บเล็ต คอม

ทุกคน · จำค่าที่เลือกไว้ในเบราว์เซอร์

04 Flow ฝั่งลูกค้า

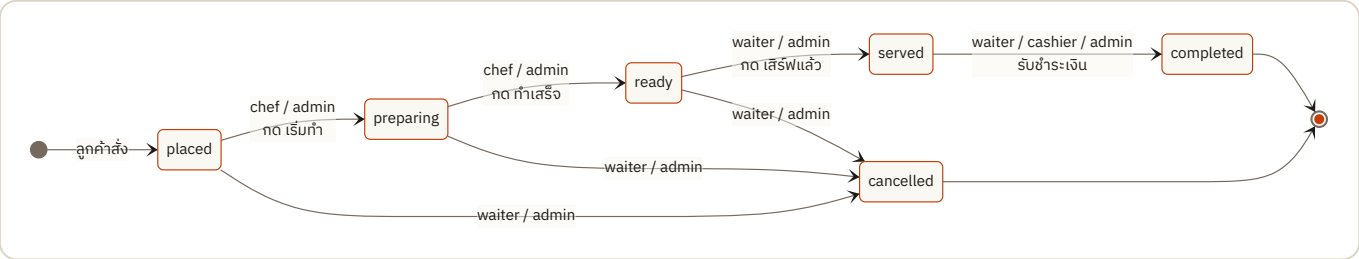
ตั้งแต่สแกน QR จนได้กินอาหาร ลูกค้าไม่ต้องสมัครหรือ login



สิ่งที่เกิดพร้อมกันตอนกดยืนยัน: ออเดอร์ขึ้นกระดานพนักงานภายใน 5 วินาทีพร้อมเสียงเตือน, สต็อกวัตถุดิบลดลง, โต๊ะ 3 เปลี่ยนเป็น "มีลูกค้า" (สองอย่างหลังทำโดย trigger)

05 วงจรชีวิตของออเดอร์

ออเดอร์มี 6 สถานะ เดินหน้าทีละขั้น ข้างลูกศรคือ role ที่กดได้

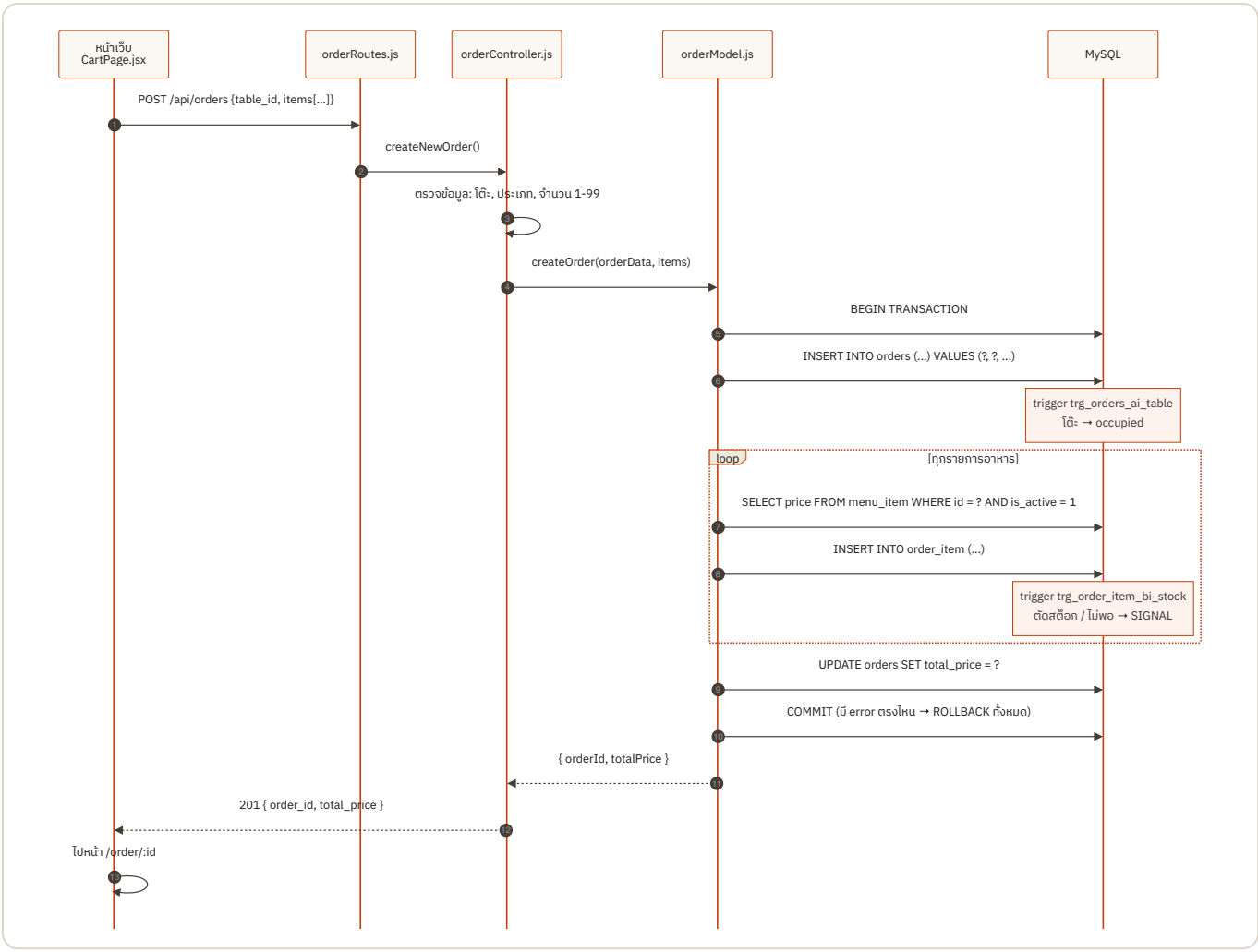


สถานะ (DB)	ชื่อบนหน้าเว็บ	อยู่คอลัมน์	ระบบทำอะไรเพิ่ม
`placed`	ใหม่ / รับออเดอร์แล้ว	ใหม่	ตัดสต็อก, โต๊ะ → occupied (trigger)
`preparing`	กำลังทำ	กำลังทำ	—
`ready`	พร้อมเสิร์ฟ	พร้อมเสิร์ฟ	—
`served`	เสิร์ฟแล้ว (รอชำระ)	เสิร์ฟแล้ว	—
`completed`	เสร็จสิ้น	แท็บประวัติ	บันทึกผู้รับเงิน เวลา เงินที่รับ · โต๊ะ → available ถ้าไม่มีออเดอร์ค้าง
`cancelled`	ยกเลิก	แท็บประวัติ	คืนสต็อก · โต๊ะ → available ถ้าไม่มีออเดอร์ค้าง

ถ้าสองคนกดพร้อมกัน: คำสั่ง UPDATE ใส่เงื่อนไข "สถานะต้องยังเป็นค่าเดิม" ไว้ใน WHERE คนที่กดทีหลังจะได้ข้อความให้ refresh แทนที่จะเขียนทับกัน ส่วนการรับเงินล็อกแถวด้วย `SELECT ... FOR UPDATE` จ่ายซ้ำไม่ได้

06 เบื้องหลัง 1 คำสั่ง: ข้อมูลไหลผ่านโค้ดยังไง

ตัวอย่าง: ลูกค้ากด "ยืนยันสั่งอาหาร" ทุก API ในระบบเดินตามเส้นทางเดียวกันนี้ route → controller → model → ฐานข้อมูล

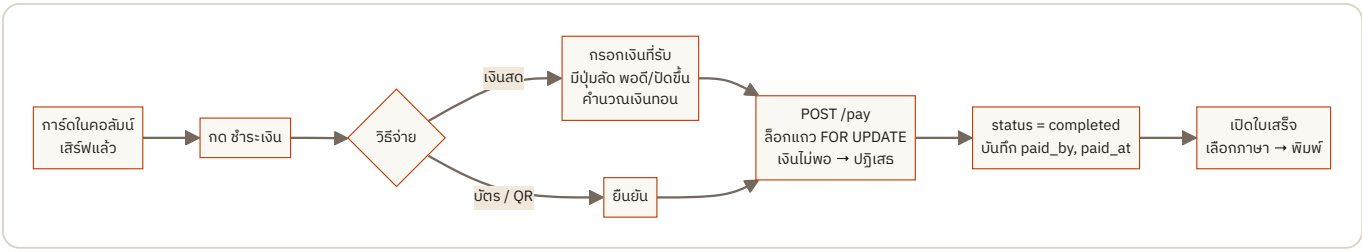


หน้าที่ของแต่ละชั้นใน backend

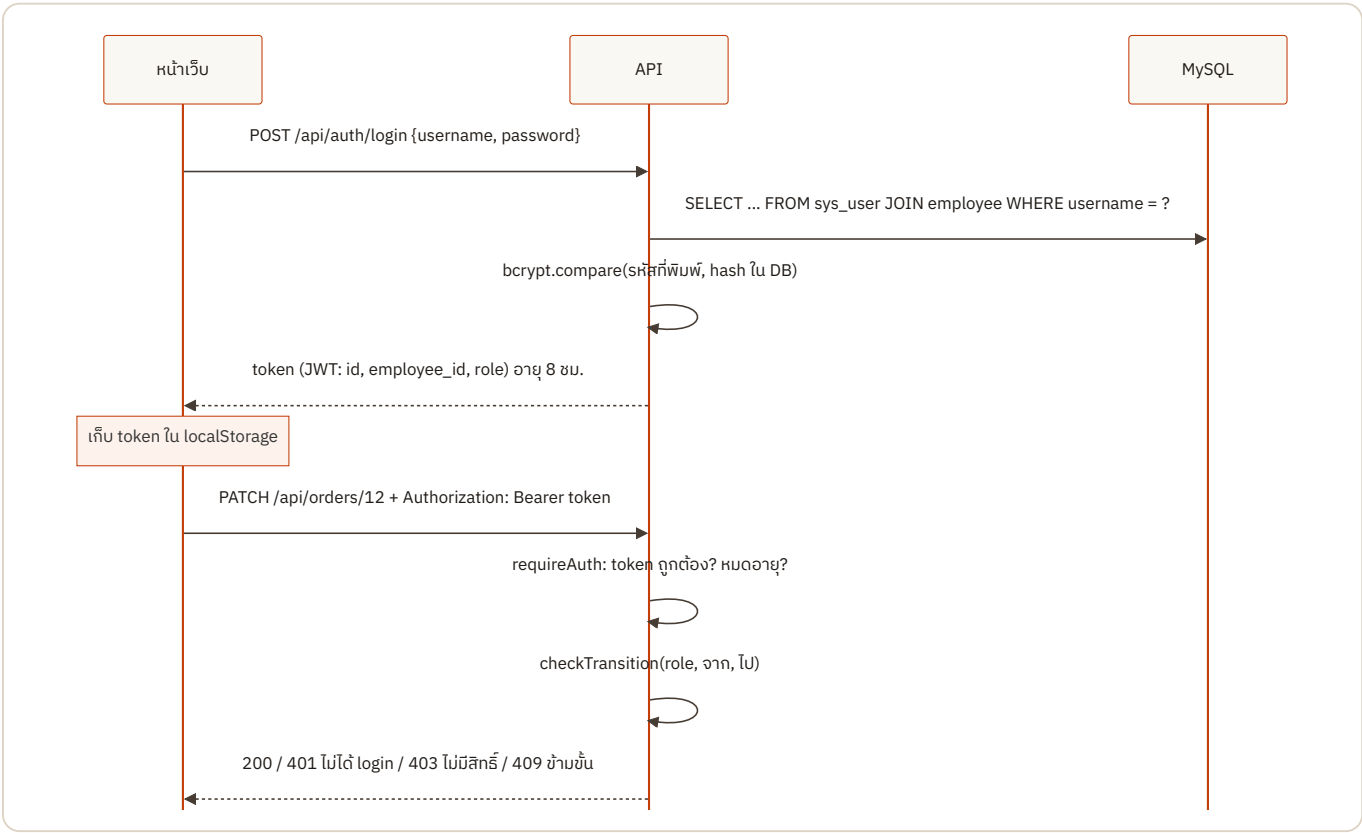
ชั้น	โฟลเดอร์	ทำอะไร	ห้ามทำอะไร
Route	`src/routes/`	จับคู่ URL กับฟังก์ชัน + ใส่ตัวตรวจสิทธิ์ `requireAuth(...)`	ไม่มี logic
Controller	`src/controllers/`	ตรวจข้อมูลที่ส่งมา, ตรวจสิทธิ์ละเอียด, เลือก status code	ไม่เขียน SQL
Model	`src/models/`	SQL ทั้งหมด (ใช้ `?` เสมอ), transaction	ไม่รู้จัก req/res
Middleware	`src/middleware/`	`auth.js` ตรวจ JWT, `errorHandler.js` แปลง error เป็น status ที่ปลอดภัย	—

07 Flow ชำระเงิน · login · ลบเมนู

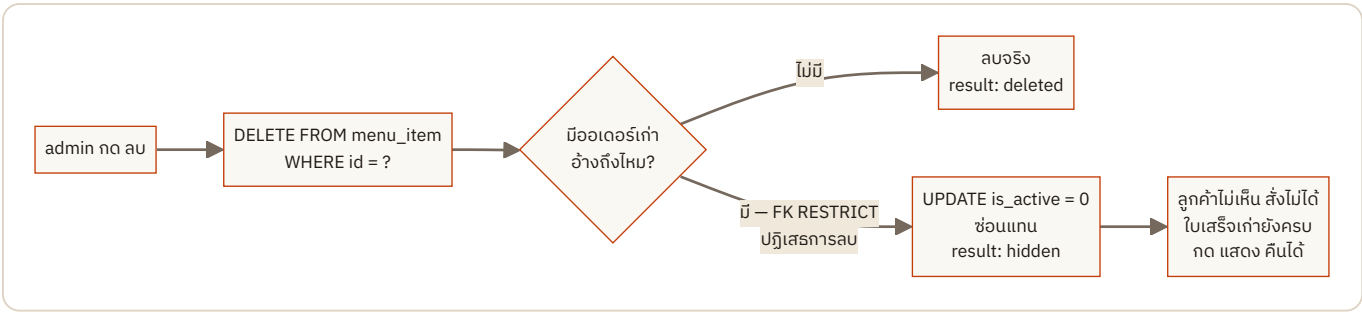
รับชำระเงิน



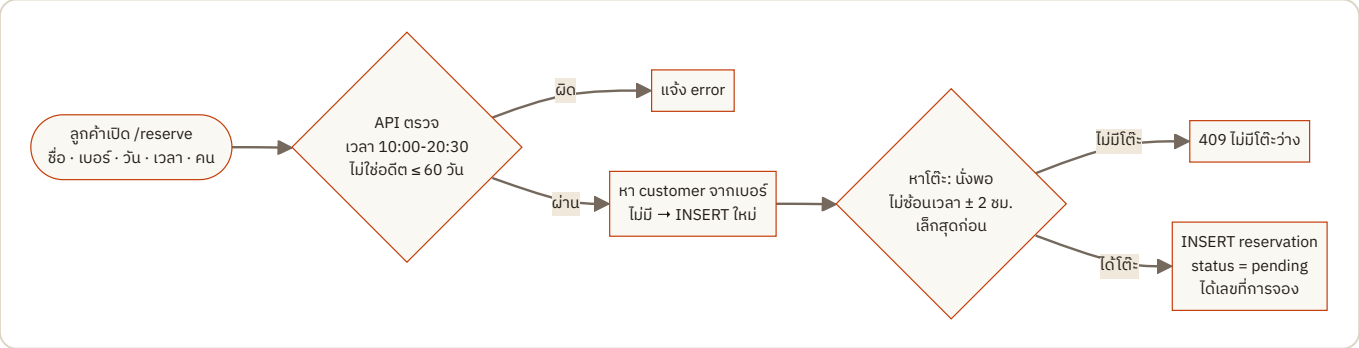
Login และการตรวจสิทธิ์ทุก request

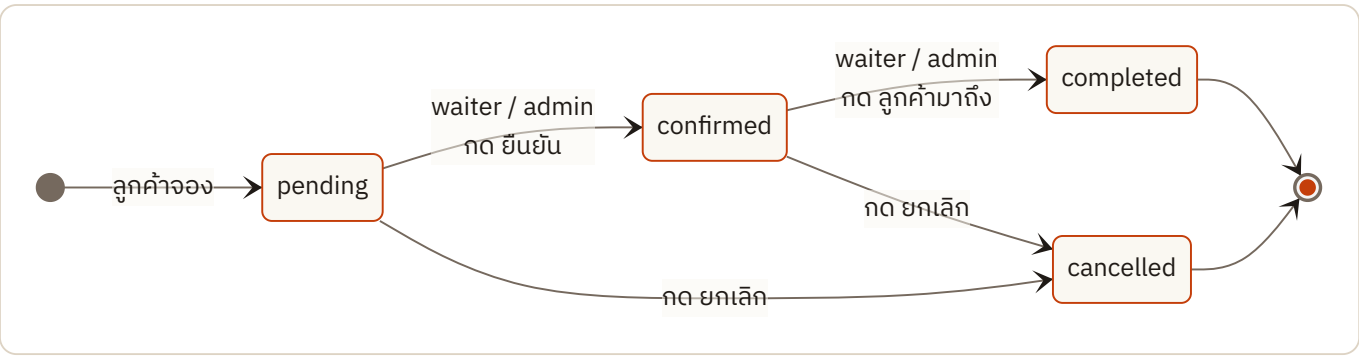


ลบเมนู: ทำไมบางเมนูลบไม่ได้



การจองโต๊ะ: ตั้งแต่ลูกค้าจองจนมานั่ง



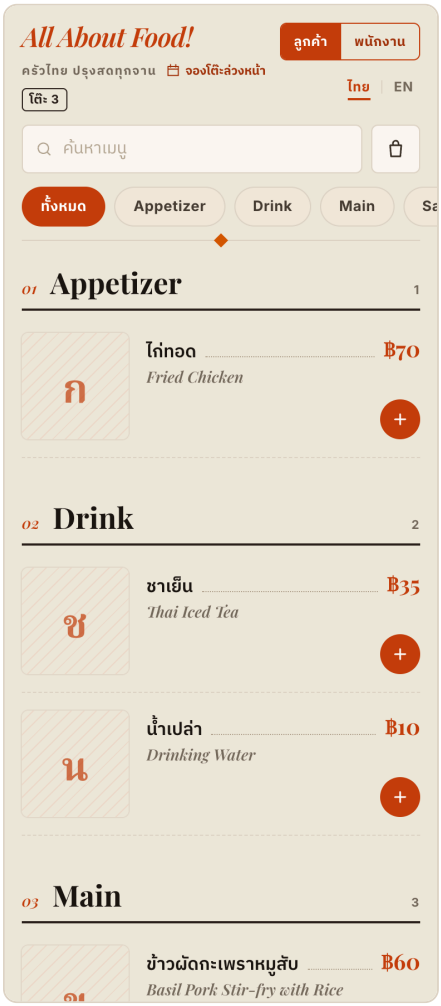


เกิดอะไรขึ้น	สถานะโต๊ะ (ทำโดย trigger)
ยืนยันการจองของ **วันนี้**	`available → reserved` (trg_reservation_au_table)
ยกเลิก / ย้ายโต๊ะ / เลื่อนวัน / ลบ	โต๊ะเดิม `reserved → available` ถ้าไม่มีการจองที่ยืนยันแล้วอื่นของวันนี้
ลูกค้ามาถึงแล้วสั่งอาหาร	`reserved → occupied` (trg_orders_ai_table)
ชำระเงินออเดอร์สุดท้ายของโต๊ะ	`occupied → available` (trg_orders_au_status)

> **กันจองซ้อน:** ตอนหาโต๊ะใช้ `SELECT ... FOR UPDATE` ใน transaction ถ้าลูกค้า 2 คนกดจองรอบเดียวกันพร้อมกัน จะได้คนละโต๊ะ ไม่ทับกัน

08 หน้าจอจริงของระบบ

ฝั่งลูกค้า (มือถือ)



หน้าเมนู: หมวด, ค้นหา, ตะกร้าลอยด้านล่าง

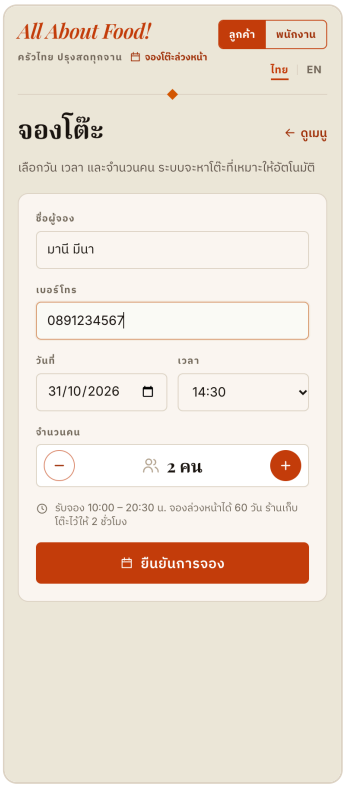


ตะกร้า: ประเภท, วิธีจ่าย, โน้ตถึงครัว



ติดตามออเดอร์: อัปเดตเองทุก 4 วินาที

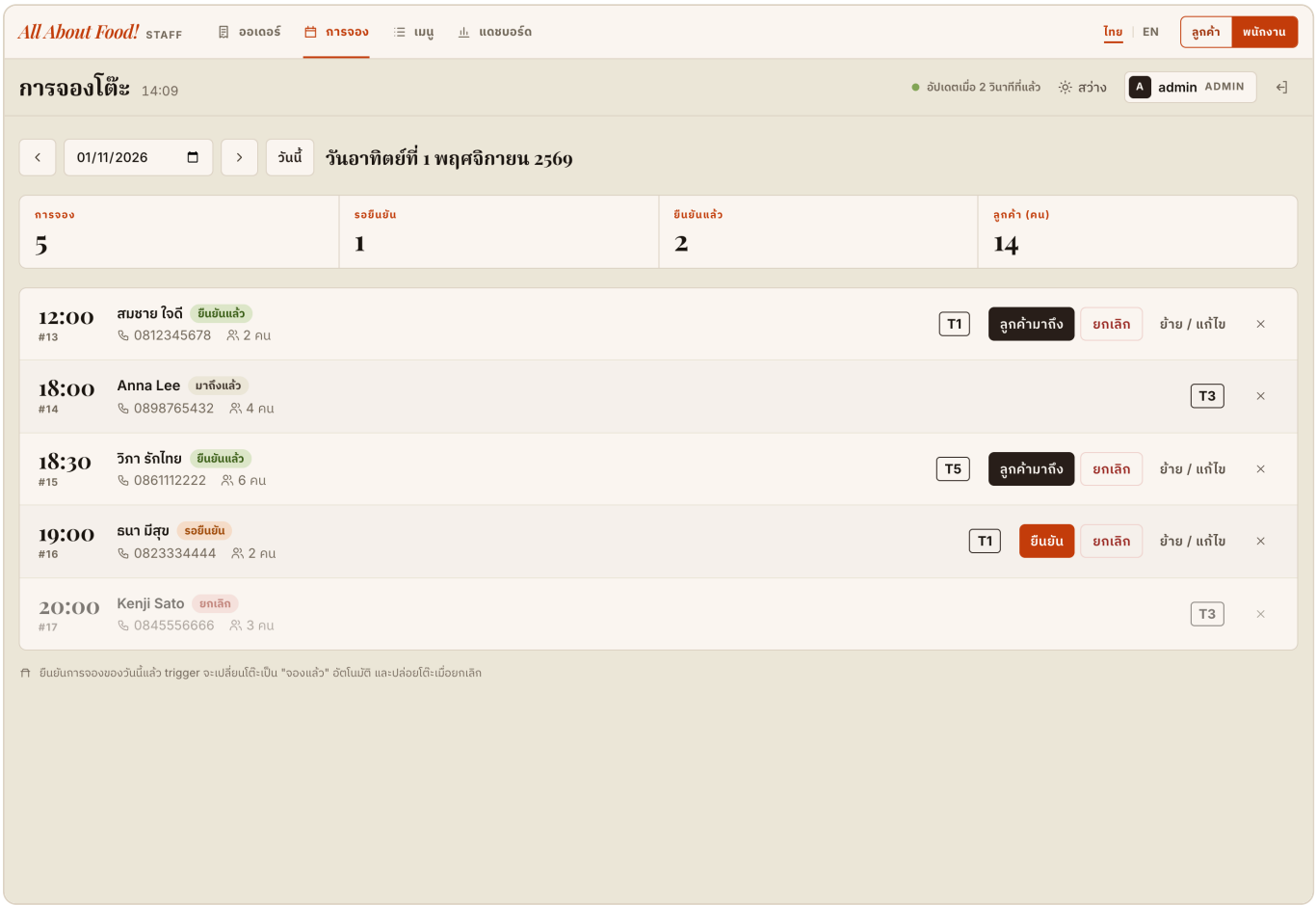
ระบบจองโต๊ะ



ลูกค้ากรอกการจอง (ลิงก์ "จองโต๊ะล่วงหน้า" อยู่ใต้ชื่อร้านทุกหน้า)

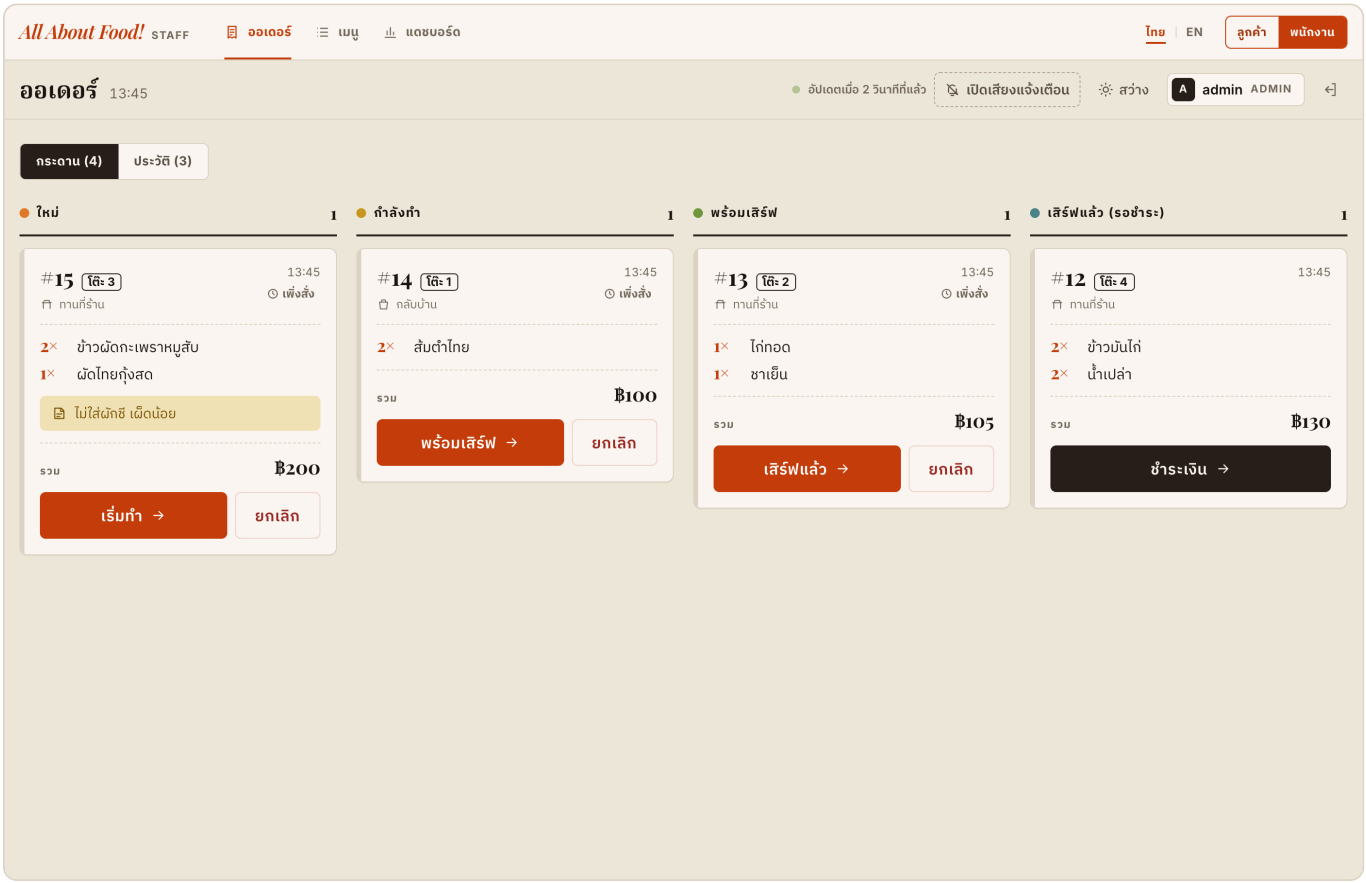


ได้เลขที่การจองและโต๊ะที่ระบบเลือกให้

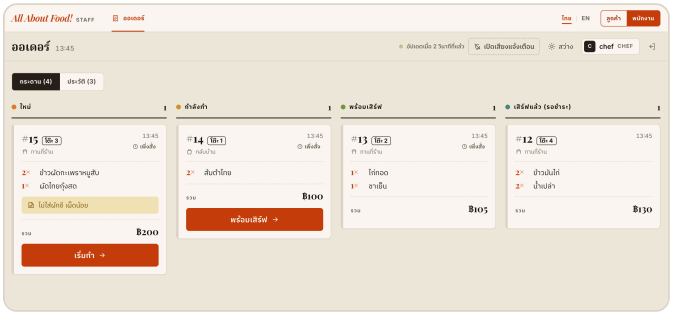


หน้าการจองของพนักงาน: เลื่อนวัน · สรุปจำนวน · ปุ่มเปลี่ยนตามสถานะ · ✕ ลบ (admin เท่านั้น)

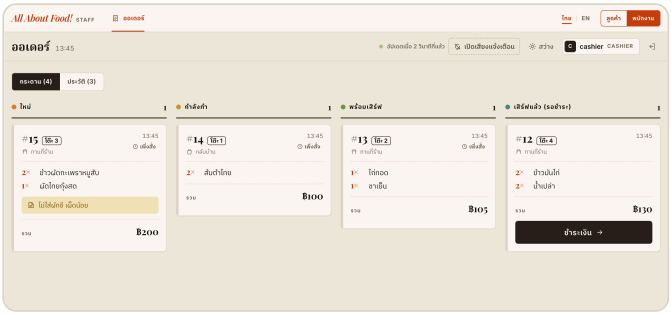
กระดานออเดอร์ของ admin (เห็นทุกปุ่ม)



ใบสั่งแบบครัว: เลขออเดอร์ โต๊ะ เวลารอ โน้ต และปุ่มขั้นถัดไปหนึ่งปุ่ม

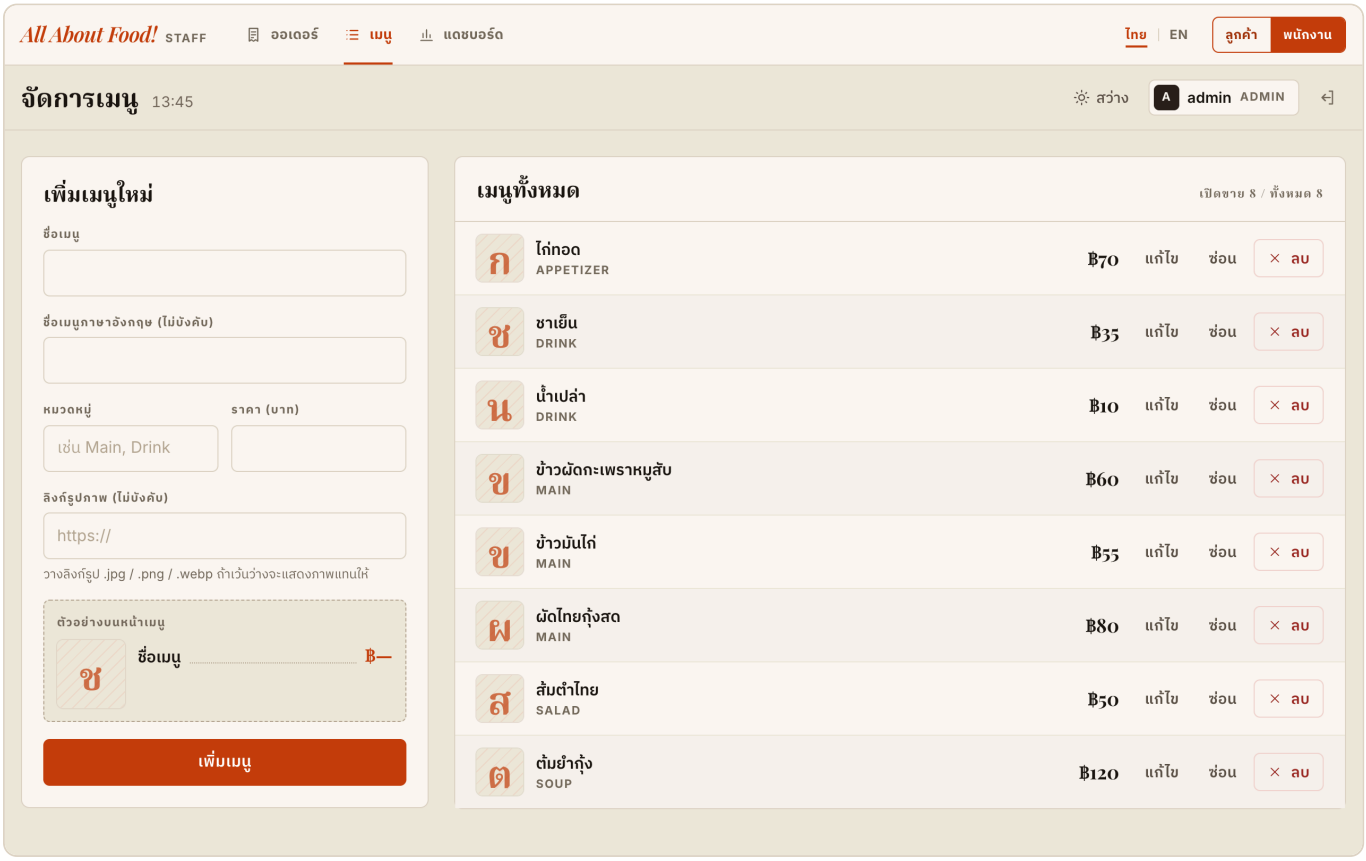


chef เห็นแค่ "เริ่มทำ" กับ "พร้อมเสิร์ฟ" ไม่มีปุ่มยกเลิกหรือชำระเงิน และไม่มีเมนูจัดการเมนู/แดชบอร์ด

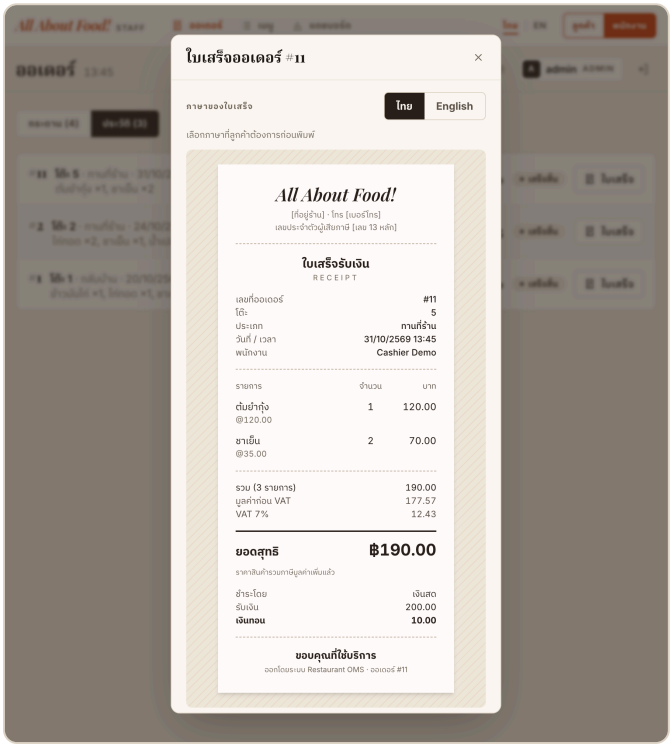


cashier เห็นแค่ปุ่ม "ชำระเงิน" ในคอลัมน์เสิร์ฟแล้ว

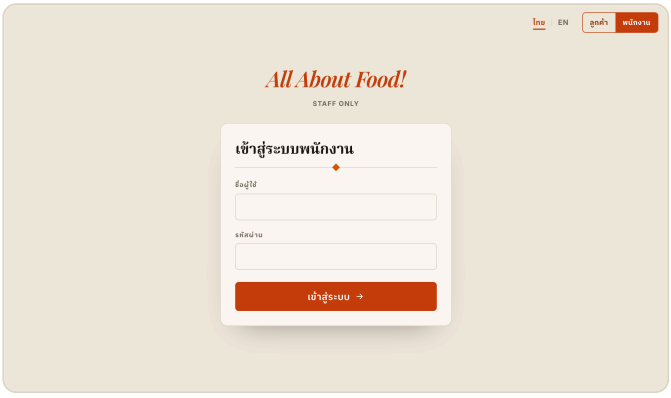
จัดการเมนู (admin)



ซ้าย: ฟอร์มเพิ่ม/แก้ไข พร้อมพรีวิว · ขวา: รายการทั้งหมด ปุ่ม แก้ไข / ซ่อน / ลบ

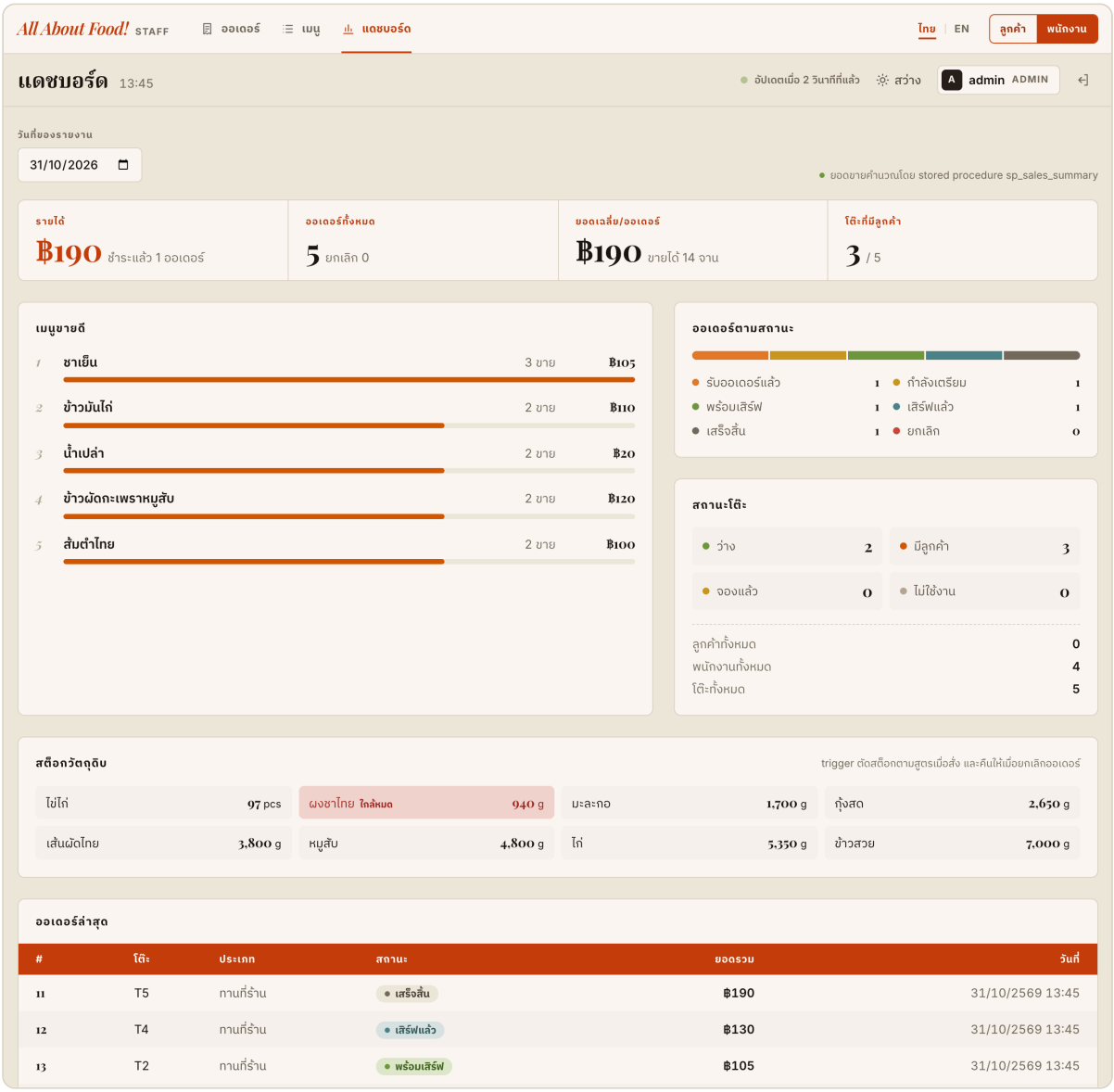


ใบเสร็จ 80 มม. เลือกภาษาได้ แยก VAT



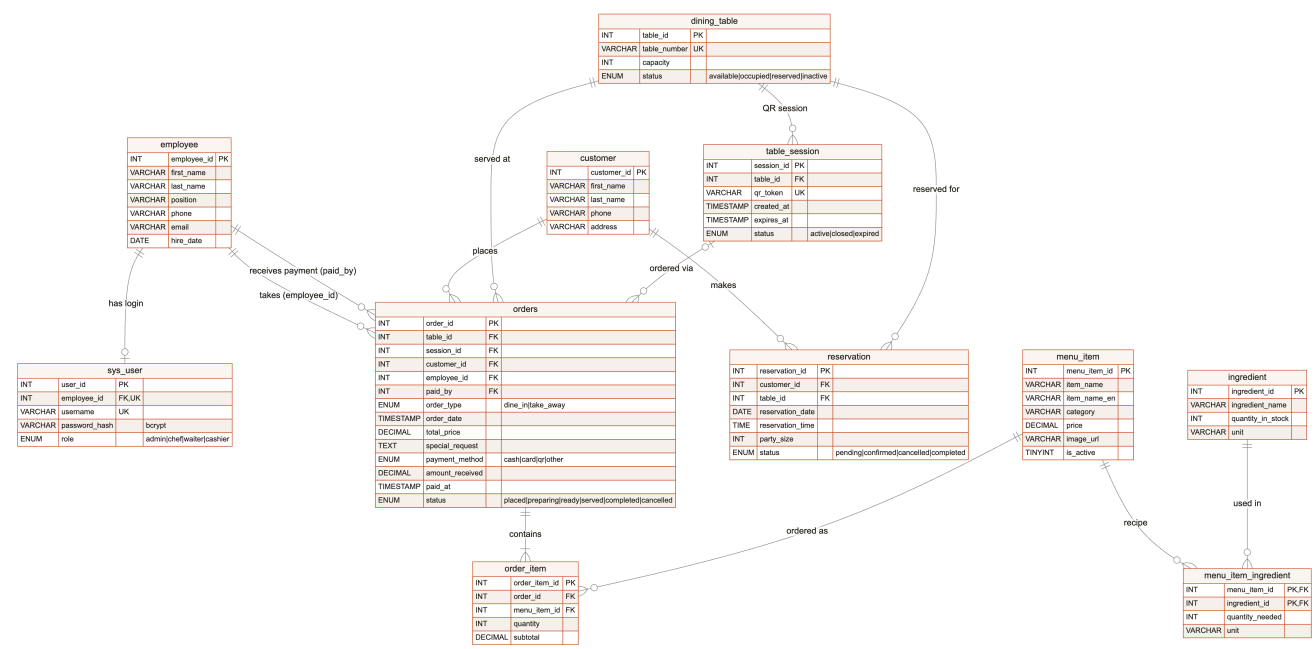
หน้า login พนักงาน

แดชบอร์ด (admin)



เลือกวันที่ → เรียก stored procedure · เมนูขายดี · ออเดอร์ตามสถานะ · สถานะโต๊ะ · สต็อกวัตถุดิบ · ออเดอร์ล่าสุด

09 ฐานข้อมูล



ER diagram สร้างจาก `database/schema.sql` · ไฟล์ `docs/er-diagram.png`

ตาราง	เก็บอะไร	ใช้ในแอปตอนนี้
`menu_item`	เมนู ราคา ชื่อไทย/อังกฤษ รูป สถานะเปิดขาย	✓
`orders`	หัวออเดอร์: โต๊ะ ประเภท สถานะ ยอดรวม การชำระเงิน	✓
`order_item`	รายการอาหารในออเดอร์ (จำนวน, ราคา ณ ตอนสั่ง)	✓
`dining_table`	โต๊ะ จำนวนที่นั่ง สถานะ	✓
`employee` / `sys_user`	พนักงาน / บัญชี login (1:1) + role	✓
`ingredient` / `menu_item_ingredient`	วัตถุดิบ / สูตรอาหาร (M:N)	✓ **ผ่าน trigger**
`customer`	ลูกค้าที่จองโต๊ะ (ค้นจากเบอร์โทร ใช้แถวเดิมถ้าเคยจอง)	✓ **ระบบจอง**
`reservation`	การจองโต๊ะ: วัน เวลา จำนวนคน สถานะ	✓ **ระบบจอง**
`table_session`	QR token ของแต่ละรอบการนั่ง	มีตาราง ยังไม่มีหน้าใช้

Trigger และ Stored procedure

ชื่อ	ทำงานเมื่อ	ทำอะไร
`trg_order_item_bi_stock`	BEFORE INSERT ON order_item	เช็คว่าวัตถุดิบพอไหม ไม่พอ → `SIGNAL` ยกเลิกทั้งออเดอร์ · พอ → ตัดสต็อกตามสูตร × จำนวน
`trg_orders_ai_table`	AFTER INSERT ON orders	ออเดอร์ทานที่ร้าน → โต๊ะ (ว่างหรือจองไว้) เป็น `occupied`
`trg_orders_au_status`	AFTER UPDATE ON orders	ยกเลิก → คืนสต็อก · ออเดอร์จบและโต๊ะไม่มีออเดอร์ค้าง → โต๊ะเป็น `available`
`trg_reservation_au_table`	AFTER UPDATE ON reservation	ยืนยันการจองของวันนี้ → โต๊ะ `reserved` · ยกเลิก/ย้าย/เลื่อน → ปล่อยโต๊ะเดิม
`trg_reservation_ad_table`	AFTER DELETE ON reservation	ลบการจองที่ยืนยันแล้วของวันนี้ → ปล่อยโต๊ะ
`sp_sales_summary(p_date)`	แดชบอร์ดเรียก `CALL`	คืน 1 แถว: รายได้, ออเดอร์ที่ชำระ, ยอดเฉลี่ย, ออเดอร์ทั้งหมด, ยกเลิก, จำนวนจาน ของวันนั้น

```
-- ลองเองใน MySQL
CALL sp_sales_summary(CURDATE());
SELECT ingredient_name, quantity_in_stock FROM ingredient;    -- สั่งอาหารแล้วรันซ้ำ ตัวเลขจะลด
SELECT table_id, status FROM dining_table;                    -- โต๊ะที่มีออเดอร์ = occupied
```

> **ข้อจำกัดของเว็บจริง:** TiDB Cloud ที่ใช้บนเว็บออนไลน์ **ไม่รองรับ trigger และ stored procedure** บนเว็บจริงจึงไม่มีการตัดสต็อก และโต๊ะไม่เปลี่ยนสถานะเอง ส่วนแดชบอร์ดใช้ SQL query สำรองที่ได้ผลเหมือนกัน (หน้าเว็บบอกไว้ว่าใช้วิธีไหน) **ตอน demo ให้ใช้ MySQL ในเครื่อง**

ไฟล์ฐานข้อมูล (รันตามลำดับ)

ไฟล์	ทำอะไร
`schema.sql`	สร้าง DB + 11 ตาราง + FK + index
`triggers.sql`	5 triggers
`procedures.sql`	`sp_sales_summary`
`seed.sql`	โต๊ะ 5 ตัว, เมนู 8 รายการ, วัตถุดิบ 8 ชนิด, สูตร 12 รายการ
`migrations/00X_*.sql`	สำหรับ DB ที่สร้างไว้ก่อนแล้ว: รันเฉพาะไฟล์ที่ยังไม่เคยรัน

10 โครงสร้างโค้ด: แก้อะไรที่ไฟล์ไหน

```
Restaurant_ahn/
├── database/            schema · triggers · procedures · seed · migrations/
├── backend/
│   ├── .env             ค่าลับทั้งหมด (ไม่อยู่ใน git) — ดูตัวอย่างใน .env.example
│   ├── src/app.js       จุดเริ่ม: ต่อ routes + CORS + error handler
│   ├── src/config/      db.js (connection pool) · permissions.js (สิทธิ์)
│   ├── src/routes/      auth · menu · order · reservation · dashboard
│   ├── src/controllers/ ตรวจข้อมูล + สิทธิ์
│   ├── src/models/      SQL ทั้งหมด
│   ├── src/middleware/  auth.js (JWT) · errorHandler.js
│   ├── scripts/         seedUsers.js · setPassword.js
│   └── tests/e2e.js     60 test cases: npm run test:e2e
├── frontend/src/
│   ├── customer/        MenuPage · CartPage · OrderStatusPage · ReservePage + components/
│   ├── staff/           LoginPage · OrdersPage · ReservationsPage · MenuAdminPage · DashboardPage
│   ├── components/      Receipt · StatusBadge · status.js (สถานะ สี สิทธิ์ฝั่งหน้าเว็บ)
│   ├── ui/              Button · Modal · Toast · Icon · fields.js (ชิ้นส่วน UI กลาง)
│   ├── services/api.js  เรียก API ทุกตัวจากที่นี่ที่เดียว
│   ├── i18n/messages.js ข้อความไทย/อังกฤษทั้งหมด
│   ├── config/shop.js   ชื่อร้าน ที่อยู่ เลขภาษี VAT บนใบเสร็จ
│   └── styles.css       สี ฟอนต์ มุมโค้ง (design tokens)
└── docs/                er-diagram · demo-test-cases · เอกสารนี้
```

อยากแก้... ไปที่ไหน

อยากทำ	ไฟล์
เปลี่ยนข้อความบนหน้าเว็บ / เพิ่มคำแปล	`frontend/src/i18n/messages.js` (แก้ทั้ง `th` และ `en`)
เปลี่ยนชื่อร้าน ที่อยู่ เลขภาษีบนใบเสร็จ	`frontend/src/config/shop.js`
เปลี่ยนสี / ฟอนต์ทั้งเว็บ	`frontend/src/styles.css` (ส่วน `@theme`)
เปลี่ยนว่า role ไหนทำอะไรได้	`backend/src/config/permissions.js` + `frontend/src/components/status.js`
เพิ่ม API ใหม่	model (SQL) → controller (ตรวจข้อมูล) → route (URL + สิทธิ์) → `services/api.js`
เพิ่มคอลัมน์ในตาราง	`schema.sql` + ไฟล์ใหม่ใน `migrations/` + model ที่เกี่ยวข้อง
เพิ่มหน้าใหม่	สร้างไฟล์ใน `customer/` หรือ `staff/` → ลงทะเบียนใน `App.jsx` → (ฝั่งพนักงาน) เพิ่มใน `NAV` ของ `StaffLayout.jsx`

11 ความปลอดภัย

เรื่อง	ระบบทำยังไง	ดูได้ที่
SQL injection	SQL ทุกคำสั่งใช้ `?` แล้วส่งค่าแยก ไม่ต่อ string จากข้อมูลผู้ใช้	`models/*.js`
รหัสผ่าน	เก็บเป็น bcrypt hash · login ผิดตอบข้อความเดียวกันทุกกรณี และใช้เวลาเท่ากัน เดาไม่ได้ว่ามี username นี้ไหม	`authController.js`
ค่าลับ	รหัส DB, JWT secret, รหัสบัญชีตัวอย่าง อยู่ใน `.env` ซึ่ง `.gitignore` ไว้	`.env.example`
Error	error ที่ไม่คาดคิดตอบแค่ "Internal server error" รายละเอียดอยู่ใน log ของ server เท่านั้น	`errorHandler.js`
สิทธิ์	ตรวจ JWT + role ที่ backend ทุก request หน้าเว็บแค่ซ่อนปุ่ม	`auth.js`, `permissions.js`
ราคา/เงิน	ราคาดึงจาก DB เสมอ · คำนวณเป็นสตางค์ (จำนวนเต็ม) กันทศนิยมเพี้ยน · รับเงินใช้ transaction + ล็อกแถว	`orderModel.js`
รูปเมนู	รับเฉพาะลิงก์ `http(s)://` กันการฝังสคริปต์	`menuController.js`
CORS	ตั้ง `CORS_ORIGIN` ให้รับเฉพาะเว็บของเรา	`app.js`

รหัสสถานะที่ API ตอบ

Code	ความหมาย	Code	ความหมาย
`200/201`	สำเร็จ / สร้างแล้ว	`404`	ไม่พบข้อมูล
`400`	ข้อมูลที่ส่งมาไม่ถูกต้อง	`409`	ข้ามขั้น, ออเดอร์ปิดแล้ว, ของหมด
`401`	ไม่ได้ login / token หมดอายุ	`500`	error ที่ไม่คาดคิด (ไม่บอกรายละเอียด)
`403`	role ไม่มีสิทธิ์		

12 รัน, deploy, demo และสิ่งที่ยังไม่มี

รันในเครื่อง (ครั้งแรก)

```
brew services start mysql                          # เปิด MySQL (ต้องเปิดก่อนทุกครั้ง)
mysql -u root -p < database/schema.sql
mysql -u root -p < database/triggers.sql
mysql -u root -p < database/procedures.sql
mysql -u root -p < database/seed.sql

cd backend && npm install && cp .env.example .env  # แก้ .env: รหัส DB, JWT_SECRET, SEED_PASSWORD
npm run seed:users && npm run dev                  # http://localhost:3000

cd ../frontend && npm install && npm run dev       # http://localhost:5173
```

> **เจอ "โหลดเมนูไม่สำเร็จ / Internal server error" ในเครื่อง** ส่วนใหญ่เพราะ MySQL ไม่ได้เปิด ให้รัน `brew services start mysql`

เว็บจริงบน cloud

ส่วน	บริการ	URL	ตั้งค่าที่ต้องรู้
Frontend	Vercel	restaurant-ahn-hclv.vercel.app	`VITE_API_URL` = URL ของ backend
Backend	Render (ฟรี)	restaurant-ahn-1.onrender.com	DB_*, `DB_SSL=true`, `JWT_SECRET`, `CORS_ORIGIN` · หลับถ้าไม่มีคนใช้ 15 นาที เปิดครั้งแรกรอ 30-60 วิ
Database	TiDB Cloud (ฟรี)	gateway01.ap-southeast-1...:4000	ไม่รองรับ trigger/procedure

Deploy อัตโนมัติ: `git push` ขึ้น `main` แล้ว Render และ Vercel build ใหม่เองภายใน 2-3 นาที ถ้าเพิ่มคอลัมน์ในฐานข้อมูล ต้องรัน migration บน TiDB (SQL Editor) **ก่อน** push

Demo ตาม rubric (30 คะแนน)

เกณฑ์	คะแนน	โชว์อะไร
ER & Schema	6	ER diagram, PK/FK ใน schema.sql, `.env` → `db.js` → model → controller → route
CRUD	7	เมนู: เพิ่ม/ดู/แก้/ซ่อน/ลบ + ลบเมนูที่เคยขายแล้ว (ซ่อนแทน) · การจอง: จอง/ดู/ยืนยัน/ย้ายโต๊ะ/ลบ
Access control	4	login ทีละ role ให้เห็นว่าปุ่มต่างกัน + `permissions.js`
Security	4	parameterized query, `.env`, error ไม่หลุด, bcrypt
SQL features	3	สั่งอาหารแล้ว SELECT สต็อก/โต๊ะ, สั่งเกินสต็อก, ยืนยันการจอง → โต๊ะ reserved, `CALL sp_sales_summary`
UI	3	ลูกค้า → พนักงาน → แดชบอร์ด, สลับภาษา, มือถือ
Professionalism	3	อธิบาย flow หน้า 6 · ปิดด้วย `npm run test:e2e` (60 passed)

ขั้นตอน demo แบบละเอียด: `docs/demo-test-cases.md`

สิ่งที่ยังไม่มี (ทำต่อได้)

เรื่อง	สถานะ
QR token ต่อรอบการนั่ง (`table_session`)	ตอนนี้ QR ใช้แค่เลขโต๊ะ ใครพิมพ์เลขโต๊ะเองก็สั่งได้
หน้าจัดการพนักงาน / วัตถุดิบ / เติมสต็อก	ยังต้องทำผ่าน SQL หรือสคริปต์
อัปเดตแบบ real-time จริง (WebSocket)	ใช้ polling ทุก 4-5 วินาทีแทน
trigger/procedure บนเว็บจริง	ต้องย้ายฐานข้อมูลจาก TiDB ไป MySQL แท้